

ACADEMY OF ECONOMIC STUDIES BUCHAREST

Faculty of Cybernetics, Statistics and Economic Informatics

Specialization: Economic Informatics (English program)

PAW PROJECT

Issue Tracking System

Theme 32. Application for managing issues
reported by QA testers and assigned to developers

Student: Munteanu Anastasia

Group: 1069, Series G

Year: 2

Academic year: 2025-2026

Course professor: Conf. univ. dr. Liviu-Adrian Cotfas

Laboratory professor: Asist. univ. dr. George Tataru

May 2026

1. Problem Definition and Application Overview

The Issue Tracking System is a desktop application written in C# with Windows Forms. It helps a development team record bugs, decide who should fix them, and follow each bug from the moment it is reported until it is closed.

In any software project, bugs are unavoidable. They are usually discovered by QA testers and then handed off to developers. Without a shared system, this information ends up scattered across emails, chat messages and informal notes, which makes it hard to know who is working on what.

This application solves that problem by storing every issue in one place, on a local SQLite database that survives between sessions. Each issue has a severity, a status, an environment, a reporter, an assignee, and a number of hours spent. Reports can be generated at any time to see how the team is doing at a glance.

1.1 Project Theme

Theme 32 from the official PAW 2026 project list: an application for managing issues reported by QA testers and assigned to developers. The required entities are:

- **Issue:** the bug or task being tracked.
- **Developer:** the person responsible for fixing the issue.
- **Resolution:** the description of how the issue was fixed, with a pull request link and resolution date.

1.2 Functional Requirements

- Add, edit and delete issues through a dedicated form with input validation.
- Add and edit developers through a separate form.
- Classify each issue by severity (Low, Medium, High, Critical).
- Track an issue through its full lifecycle (Open, In Progress, Ready for QA, Testing, Reopened, Closed).
- Specify the environment where the issue occurs (Local, Dev, Staging, Production).
- Record which QA tester reported the issue and which developer is assigned to it.
- Track time spent on each issue, with one decimal place.
- Persist all data in a local SQLite database between application runs.
- Export issues to CSV or JSON files.
- Import issues from CSV or JSON files through the menu or via drag-and-drop.
- Generate three types of reports (issues per developer, severity distribution, status summary) and save any report as a text file.

- Preview a printable list of issues before sending it to the printer.
- Show inline error messages with an ErrorProvider when validation fails.
- Display a summary card of the currently selected issue, with severity color-coded.

1.3 Non-functional Requirements

- Runs on any Windows machine with .NET Framework 4.8 installed.
- Uses only standard Windows Forms controls, so it looks like a normal desktop app.
- All data persists across application restarts.
- Menus support Alt-key shortcuts (for example Alt+F opens the File menu).
- Users can act on issues in four different ways: main menu, toolbar, right-click context menu, or drag-and-drop.
- Status bar gives immediate feedback after every action.

1.4 Technologies Used

- **Programming language:** C#.
- **Framework:** .NET Framework 4.8.
- **UI library:** Windows Forms.
- **Database:** SQLite, accessed through System.Data.SQLite (ADO.NET pattern).
- **JSON serialization:** JavaScriptSerializer from System.Web.Extensions.
- **Binary serialization:** BinaryFormatter.
- **CSV serialization:** manual implementation with proper escaping of commas and quotes.
- **IDE:** Microsoft Visual Studio 2026 Community Edition.
- **Version control:** Git, with the project pushed to a private GitHub repository.

2. Application Architecture

The code is organized in layers, with each layer doing one thing. The user interface depends on view models and services, services depend on models, and models do not depend on anything else. Dependencies always go in one direction, which keeps the project easy to navigate.

2.1 Project Structure

The solution contains two projects:

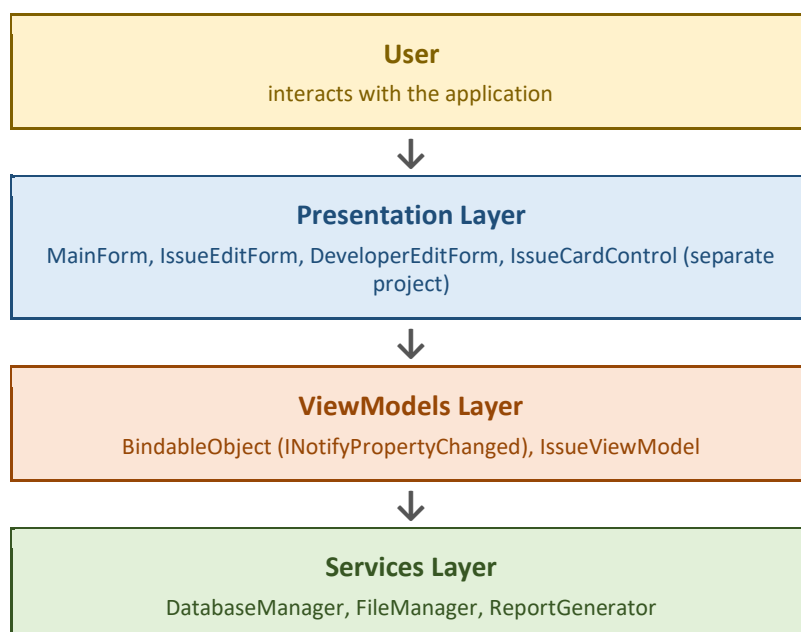
- **IssueTracker:** the main project, with models, view models, services, forms and the SQLite database access code.
- **IssueTracker.Controls:** a separate Class Library that hosts the IssueCardControl UserControl. Keeping the control in its own assembly satisfies the rubric requirement to have a UserControl defined in a different project.

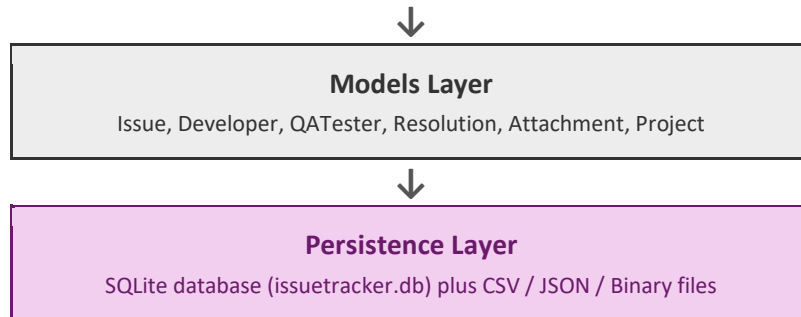
Inside the IssueTracker project, the folders are:

- **Models:** plain data classes (Issue, Developer, QATester, Resolution, Attachment, Project).
- **ViewModels:** wrappers around model objects that implement INotifyPropertyChanged for two-way binding.
- **Services:** DatabaseManager, FileManager, ReportGenerator.
- **Exceptions:** custom exception classes for domain-specific errors.
- **Forms:** MainForm, IssueEditForm, DeveloperEditForm.

2.2 Layered Architecture Diagram

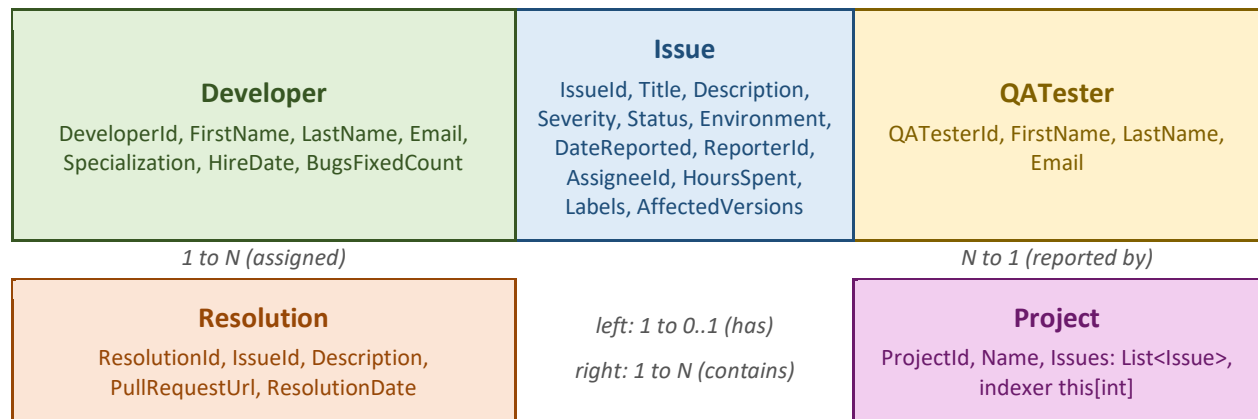
The diagram below shows how the layers stack on top of each other. The arrows show direction of dependency: each layer uses only the layers below it.



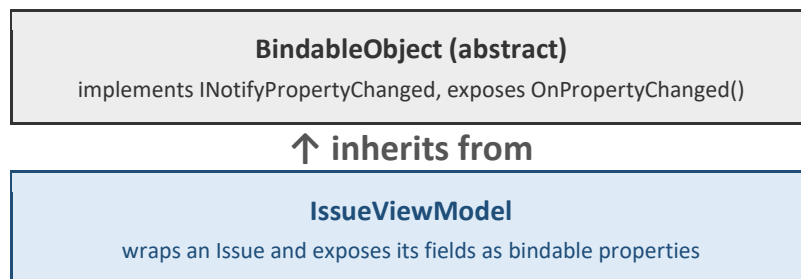


2.3 Class Diagram

The diagram below shows the main domain classes, their key fields and the relationships between them. The numbers next to relationship labels show cardinality: 1 means one and N means many.



Below the domain classes, the data binding layer is built around the following inheritance chain:



2.4 Relationships Between Entities

- **Issue and Developer (Assignee)**: each issue references one developer through AssigneeId. The same developer can be assigned to many issues, so this is a many-to-one relationship.
- **Issue and QATester (Reporter)**: each issue references one QA tester through ReporterId. One QA tester can report many issues.
- **Issue and Resolution**: each issue can have at most one resolution. The resolution carries the IssueId as a foreign key, so this is one-to-one.

- **Project and Issue:** a project contains a list of issues. The Project class has an indexer and an AddIssue method to manage this collection.
- **IssueViewModel and Issue:** composition. The view model wraps an issue instance and exposes its fields as bindable properties.
- **BindableObject and IssueViewModel:** inheritance. IssueViewModel extends BindableObject to inherit the INotifyPropertyChanged implementation.

3. Description of Classes

This chapter goes through each class in the application, grouped by layer.

3.1 Models

Issue

Represents a bug or task in the system. The class is marked [Serializable] so it can be saved using BinaryFormatter. It uses private fields with public properties, and the setters contain basic validation (title cannot be empty, hours cannot be negative).

Key fields:

- **IssueId (int):** unique identifier.
- **Title (string):** short summary of the bug, required.
- **Description (string):** longer text description.
- **Severity (Severity):** Low, Medium, High, or Critical.
- **Status (IssueStatus):** current lifecycle stage.
- **Environment (IssueEnvironment):** where the bug shows up.
- **DateReported (DateTime):** when it was first reported.
- **ReporterId (int):** foreign key to QATester.
- **AssigneeId (int):** foreign key to Developer.
- **HoursSpent (double):** time spent so far, in hours.
- **Labels (string[]):** tags used for grouping.
- **AffectedVersions (int[]):** version numbers affected by the bug.

Developer

Represents a developer who can be assigned to fix issues.

Key fields:

- **DeveloperId, FirstName, LastName, Email:** basic identity fields.
- **Specialization (Specialization):** Backend, Frontend, FullStack, Mobile, or DevOps.
- **HireDate (DateTime):** when the developer joined.
- **BugsFixedCount (int):** running count of resolved issues.
- **TotalDeveloperCount (static):** shared counter that increments every time a new Developer is created. This is the example of static field requested by the lab rubric.
- **MAX_LANGUAGES (const):** compile-time constant, used here as the example of const field required by the rubric.

Developer overrides ToString() to return a friendly label like "Andrei Popescu (Backend)". This is what shows up automatically when a Developer is added to a ComboBox.

QATester

Represents a QA tester. Has `QATesterId`, `FirstName`, `LastName`, `Email`. Overrides `ToString()` to show "FirstName LastName (QA)".

Resolution

Represents the fix applied to an issue. Has `ResolutionId`, `IssueId` (foreign key), `Description`, `PullRequestUrl`, `ResolutionDate`.

Attachment

Represents a file attached to an issue. Has `AttachmentId`, `IssueId`, `FileName`, `ContentType`, `UploadDate`.

Project

Represents the parent project that contains issues. Has `ProjectId`, `Name`, and a `List<Issue>` field. It defines an indexer (`this[int i]`) for accessing issues by position, and an `AddIssue(Issue)` method.

3.2 Enumerations

- **Severity:** Low, Medium, High, Critical.
- **IssueStatus:** Open, InProgress, ReadyForQA, Testing, Reopened, Closed. These six values cover the full lifecycle of an issue.
- **IssueEnvironment:** Local, Dev, Staging, Production. Named `IssueEnvironment` (not just `Environment`) to avoid a name clash with `System.Environment`.
- **Specialization:** Backend, Frontend, FullStack, Mobile, DevOps.

3.3 ViewModels**BindableObject (abstract)**

Abstract base class that implements `System.ComponentModel.INotifyPropertyChanged`. It exposes a protected `OnPropertyChanged` method that raises the `PropertyChanged` event automatically, using the `[CallerMemberName]` attribute so the caller does not have to pass the property name. Derived view-model classes call `OnPropertyChanged()` inside their setters, and WinForms data binding picks up the change automatically.

IssueViewModel

Wraps an `Issue` instance and exposes each of its fields as a bindable property. Every setter assigns the new value to the inner `Issue` and then calls `OnPropertyChanged()`. This enables two-way data binding: when the user types in a textbox, the view model and the underlying `Issue` both update; when code changes the issue, the controls refresh automatically.

This class is used inside `IssueEditForm`. The form sets up bindings with `Bindings.Add(...)` for each control. The `GetIssue()` method returns the underlying `Issue`, which the form passes back to `MainForm` when the user clicks Save.

3.4 Services

DatabaseManager

Handles all SQLite database access. It opens and closes the connection, makes sure the schema exists on first run (via `EnsureDatabase`), and exposes CRUD methods. It uses the classic ADO.NET pattern: `SQLiteConnection` plus `SQLiteCommand` with parameterized queries, which avoids SQL injection.

Key methods:

- **EnsureDatabase():** creates the Issues and Developers tables if they do not exist yet.
- **GetAllIssues, GetAllDevelopers:** return the full table content as a List.
- **InsertIssue, InsertDeveloper:** add a new record with parameterized SQL.
- **UpdateIssue, UpdateDeveloper:** update an existing record by ID.
- **DeleteIssue:** remove an issue by ID.

FileManager

Handles serialization and deserialization in three formats. CSV uses a manual implementation with proper escaping of commas and quotes inside text fields. JSON uses `JavaScriptSerializer` from `System.Web.Extensions`. Binary uses `BinaryFormatter`.

Key methods:

- **SaveIssuesToCsv, LoadIssuesFromCsv:** CSV export and import.
- **SaveIssuesToJson, LoadIssuesFromJson:** JSON export and import.
- **SaveIssuesToBinary, LoadIssuesFromBinary:** binary export and import.

ReportGenerator

Builds plain-text reports from the list of issues. Each method returns a formatted string that can be shown in a textbox or saved to a file.

Key methods:

- **IssuesPerDeveloper:** groups issues by assignee and counts how many each developer is handling.
- **SeverityDistribution:** counts how many issues fall into each severity level and shows percentages.
- **StatusSummary:** counts issues per status.
- **SaveReportToTxt:** writes any report string to a text file.

3.5 Exceptions

Custom exception classes inherit from `System.Exception` and carry domain-specific error messages.

- **IssueNotFoundException:** thrown when an issue lookup by ID fails.
- **DuplicateIdException:** thrown when trying to insert an entity whose ID already exists.

- **InvalidStatusTransitionException:** thrown when an invalid status change is attempted.

3.6 Forms

MainForm

The main window of the application. It hosts:

- **DataGridView (dgvIssues):** shows the list of all issues. It is bound to a `BindingList<Issue>`, so it refreshes by itself when an issue is added, edited or deleted.
- **MenuStrip:** contains File, Edit, Reports, Manage and Help menus, with Alt-key shortcuts (&File, &Save, and so on).
- **ToolStrip:** quick buttons for New, Save and Load.
- **StatusStrip:** a single label that shows confirmation messages at the bottom.
- **ContextMenuStrip (cmsIssueGrid):** right-click menu on the grid with Edit Issue, Delete Issue, Change Status and Copy ID.
- **IssueCardControl (issueCard):** the reusable card from the separate project, used to show details of the selected issue.

The form also implements drag-and-drop import: `AllowDrop` is set to true, and the `DragEnter` and `DragDrop` events validate the dropped file and load it.

IssueEditForm

Secondary form for creating or editing one issue. It uses two constructors, following the pattern from the Cotfas labs:

- **IssueEditForm(developers, qaTesters):** for creating a new issue.
- **IssueEditForm(existing, developers, qaTesters):** for editing an existing one. Pre-fills the controls with the existing data.

Full data binding is set up with `Bindings.Add(...)` between the controls and the `IssueViewModel`. Input validation runs through `Validating` events on the title and hours-spent fields, and errors are displayed by an `ErrorProvider`.

DeveloperEditForm

Secondary form for creating or editing one developer. Follows the same two-constructor pattern. Validation runs on the first name and last name fields.

3.7 User Control (separate project)

IssueCardControl

A reusable `UserControl` defined in its own Class Library project (`IssueTracker.Controls`), which compiles to a separate `.dll`. The main `IssueTracker` project references this library and drops the control onto `MainForm`.

The control shows the selected issue's key information (ID, Title, Severity, Status, Hours) as a small card. The severity label is color-coded: dark red for Critical, orange-red for High, dark orange for Medium, sea green for Low.

Public methods:

- **DisplayIssue(int id, string title, string severity, string status, double hours):** updates the card with the given data.
- **ClearCard():** resets the card to its empty state.

The methods only accept primitive parameters (int, string, double), so the control does not depend on the main project's model classes. This keeps the two projects loosely coupled.

4. Description of Application Functions

This chapter walks through every action the user can take, grouped by feature area.

4.1 Issue Management

Create a new issue

Triggered from File then New Issue, the toolbar New button, or right-click on the grid. It opens IssueEditForm with empty fields. After the user fills everything in and clicks Save, the issue is added to the BindingList (which refreshes the grid by itself) and inserted into the SQLite database.

Edit an existing issue

Triggered by double-clicking a row, by right-click then Edit Issue, or from the Edit menu. The form opens with the issue's data already filled in, and the Issue ID is disabled to prevent it from being changed. On Save, the database is updated and the grid is refreshed using BindingList.ResetItem.

Delete an issue

Triggered by right-click then Delete Issue or from the Edit menu. A confirmation MessageBox shows the issue title. If the user confirms, the issue is removed from both the database and the in-memory list.

Copy issue ID

Right-click then Copy ID. Places the issue's ID on the Windows clipboard, ready to be pasted into a Git commit message or anywhere else. The status bar confirms the action.

4.2 Developer Management

Add a new developer

From Manage then Add Developer. Opens DeveloperEditForm with empty fields. On Save, the developer is added to the in-memory list and inserted into the database.

Edit an existing developer

From Manage then Edit Selected Developer. The application prompts for the developer ID, looks the developer up, and opens DeveloperEditForm with that developer's data pre-filled.

4.3 File Operations

Save issues to file

From File then Save (or the toolbar Save button). Opens a SaveFileDialog that lets the user choose between CSV and JSON formats. Saves all issues currently in the grid to the selected file.

Load issues from file

From File then Load (or the toolbar Load button). Opens an OpenFileDialog for CSV or JSON files. The contents replace the current in-memory list. The database is not changed by this action, since loading is meant for previewing imported data before deciding what to do with it.

Drag-and-drop import

The user drags a CSV or JSON file from File Explorer onto the IssueTracker window. The DragEnter event accepts the file (showing a copy cursor), and the DragDrop event reads the file and imports its contents into the list.

4.4 Reports

Issues per Developer

From Reports then Issues per Developer. Groups all issues by their assignee and lists each developer with the number of issues currently assigned. The report opens in a modal window with a Save to TXT button that exports the text to a file.

Severity Distribution

From Reports then Severity Distribution. Counts how many issues fall into each severity level and shows percentages of the total.

Status Summary

From Reports then Status Summary. Counts issues per status (Open, In Progress, Ready for QA, Testing, Reopened, Closed).

4.5 Print Preview

From File then Print Preview. Opens a PrintPreviewDialog that renders a printable list of all issues. The first page has a title with the generation timestamp, then a header row (ID, Title, Severity, Status, Hours), followed by one issue per line. Pagination is handled by setting e.HasMorePages to true when the current page runs out of space, so the print framework asks for another one. From the preview, the user can print directly or save as PDF through the standard Windows print dialog.

4.6 Issue Card (live preview)

When the user clicks a row in the grid, the SelectionChanged event fires and updates the IssueCardControl on the right side of the form. The card shows the selected issue's ID, title, severity (color-coded), status, and hours. When no row is selected (for example, after the only issue is deleted), ClearCard() resets the card to its default empty state.

5. Menu Description

This chapter describes each menu, toolbar button, and context-menu entry. Menu items with an ampersand (&) before a letter support Alt-key shortcuts. For example, Alt+F opens the File menu, then Alt+N selects New Issue.

5.1 Main Menu (MenuStrip)

Menu	Item	Action
&File	&New Issue	Opens IssueEditForm to create a new issue.
	&Save	Exports current issues to a CSV or JSON file.
	&Load	Imports issues from a CSV or JSON file.
	&Print Preview	Opens the print preview with a printable issue list.
	E&xit	Closes the application.
&Reports	Issues per Developer	Shows how many issues are assigned to each developer.
	Severity Distribution	Shows the count and percentage of issues per severity level.
	Status Summary	Shows the count of issues per status.
&Manage	Add Developer	Opens DeveloperEditForm to create a new developer.
	Edit Selected Developer	Prompts for a developer ID and opens the edit form.
&Help	About	Shows project information (theme, group, year).

5.2 Toolbar (ToolStrip)

Button	Action
New	Same as File then New Issue. Opens the issue editor for a new issue.
Save	Same as File then Save. Opens the export dialog.
Load	Same as File then Load. Opens the import dialog.

5.3 Context Menu (right-click on grid)

Item	Action
Edit Issue	Opens the selected issue in IssueEditForm.
Delete Issue	Deletes the selected issue after confirmation.
Change Status	Placeholder for a future dropdown to change status directly without opening the editor.
Copy ID	Copies the selected issue's ID to the clipboard.

5.4 Status Bar (StatusStrip)

A single label (lblStatus) at the bottom of the main window shows the result of the most recent action. Typical messages are "Added issue #5", "Updated developer #2", "Imported 12 issues from dropped file", or "Copied ID 3 to clipboard". This gives the user immediate confirmation that an action succeeded.

6. Conclusions

The Issue Tracking System covers all the main Windows Forms topics from the PAW course. The application is split into clear layers, the database access uses ADO.NET against SQLite, the forms validate input and surface errors with an ErrorProvider, the menus support keyboard shortcuts, and the data flows two ways thanks to the BindableObject base class.

Beyond the basics, the project includes three serialization formats (CSV, JSON, binary), three types of reports, a print preview that paginates automatically, a reusable user control hosted in a separate Class Library, and drag-and-drop file import.

There are a few obvious directions for future work:

- Add a custom GDI+ chart to visualize the severity distribution graphically.
- Migrate the data access layer to Entity Framework Core with code-first migrations.
- Introduce role-based access, so QA testers and developers see different views.
- Sync with a remote API so that multiple machines can share the same data.
- Send email notifications when an issue is assigned or its status changes.